Documentación por Unidad (1 a 6) — Problema, Códigos, Solución y Cierre

> NOTE
>
> Cada unidad sigue el mismo formato: **Introducción breve** → **Temas expuestos** → **Orden de ejecución** → **Explicación de los códigos** → **Cierre**. Donde dice *(Insertar captura...)* va la imagen del código o de la consola.

> IMPORTANT
>
> En **todas** las unidades hay un script que **reproduce el problema** y otro script que **lo soluciona**. La tabla "Orden de ejecución" de cada unidad indica exactamente qué correr y en qué orden.

Unidad	Script que muestra el problema	Script que lo soluciona
1	`sync_service.sh` + `audit_syscalls.sh`	`reparar_servicio.sh` (y `restaurar_falla.sh` para repetir la demo)
2	`demostracion tpo.sh` + `carrera_escritura.sh`	`seguro.c` (compilado y ejecutado por `demostracion_tpo.sh`)
3	`ejecutar_tpo.sh` (ejecuta modo `F` First-Fit)	`ejecutar_tpo.sh` (ejecuta modo `B` Best-Fit en la misma corrida)
4	`ejecutar_tpo.sh` (ejecuta modo `F` contiguo)	`ejecutar_tpo.sh` (ejecuta modo `I` Inodos ext4 en la misma corrida)
5	`ejecutar_tpo.sh` (ejecuta `sin_bufer`)	`ejecutar_tpo.sh` (ejecuta `con_bufer` en la misma corrida)
6	`ejecutar_tpo.sh` (bloqueo de puerto en `servera` y modificación de `pagos_empresa.sh`)	`ejecutar_tpo.sh` (apertura en `serverb`, usuario `/etc/shadow`, ACL y bloqueo del virus con `sha256sum -c`)

Unidad 1 — Llamadas al Sistema (Syscalls) y Auditoría con `strace`

Introducción breve

En la empresa **SecurePay FinTech Solutions**, cada medianoche se ejecuta en un servidor Red Hat el script `sync_service.sh`, encargado de conciliar las transacciones del día. Al pasar a producción, el proceso empezó a cortarse a los pocos segundos mostrando únicamente el mensaje `[ERROR FATAL 101]: Fallo critico en el motor transaccional`. El mensaje no indica qué falló, y las áreas se culpaban entre sí (desarrollo sospechaba de la base de datos y el proveedor decía que el problema era el servidor). El objetivo es encontrar la causa real **sin modificar el código del servicio**, observando qué le pide el programa al Kernel.

Temas expuestos

- **Modo Usuario y Modo Kernel:** los programas comunes (Modo Usuario) no pueden acceder directamente al disco ni al hardware; tienen que pedírselo al Kernel, que es el único que trabaja con privilegios completos.
- **Llamadas al sistema (syscalls):** son la forma en que un programa le pide un servicio al Kernel (abrir un archivo, leerlo, escribir en pantalla). Si la operación falla, el Kernel devuelve `-1` y un código de error (por ejemplo `ENOENT` = el archivo no existe, `EACCES` = permiso denegado).
- **`strace`:** herramienta de Linux que intercepta y muestra todas las llamadas al sistema que hace un programa, con su resultado, sin necesidad de tocar su código.
- **Descriptores de archivo:** `0` entrada estándar, `1` salida estándar y `2` salida de errores (por eso el mensaje de error usa `>&2`).

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./sync_service.sh`	El servicio falla con `[ERROR FATAL 101]` (el problema).
2	`./audit_syscalls.sh`	`strace` revela la causa: `faccessat2(... "/etc/secure_app/transactions.dat" ...) = -1 ENOENT`.
3	`./reparar_servicio.sh`	Se crea la carpeta y el archivo faltante; el servicio termina con código `0` y `strace` muestra `= 0`.
4 (opcional)	`./restaurar_falla.sh`	Vuelve a dejar el servidor con el problema para repetir la demostración.

> **WARNING**
>
> El comando `chmod` solo funciona sobre archivos que **ya existen**. Por eso la solución no es solamente `chmod 644`: primero hay que crear la carpeta `/etc/secure_app` y el archivo `transactions.dat`, que es lo que hace `reparar_servicio.sh`.

Código `sync_service.sh` (el servicio que falla)

(Insertar captura de `sync_service.sh`)

Desglose línea por línea

- **`#!/bin/bash`**: indica al sistema operativo que el archivo debe ejecutarse con el intérprete Bash.
- **`DATA_FILE="/etc/secure_app/transactions.dat"`**: guarda en la variable `DATA_FILE` la ruta del archivo de transacciones que el servicio necesita leer.
- **`echo "[INFO] Iniciando el servicio de conciliacion..."`**: muestra en pantalla un mensaje informativo indicando que el servicio comenzó.
- **`if [ ! -r "$DATA_FILE" ]; then`**: antes de continuar, verifica una condición. `-r` comprueba si el archivo existe y se puede leer; el `!` niega esa condición. En conjunto significa: "*si el archivo NO se puede leer*" (porque no existe o porque no hay permiso de lectura).
- **`echo "[ERROR FATAL 101]: ..." >&2`**: si no se puede leer, muestra el mensaje de error. `>&2` envía el texto por la salida de errores (descriptor 2).
- **`exit 101`**: termina el script en ese momento devolviendo el código `101`, que indica que finalizó con error.
- **`fi`**: cierra el bloque `if`.

- `cat "$DATA_FILE"`: si el archivo sí se pudo leer, muestra su contenido en pantalla.
- `exit 0`: termina el script devolviendo `0`, que en Linux significa que finalizó correctamente.

En resumen

El script guarda la ruta del archivo de transacciones, avisa que está iniciando y verifica si puede leer ese archivo. Si no puede, muestra el Error Fatal 101 y se detiene con código `101`. Si puede, muestra las transacciones y termina con código `0`. El problema es que el mensaje de error no dice **por qué** no pudo leer el archivo.

Código `audit_syscalls.sh` (el diagnóstico)

(Insertar captura de `audit_syscalls.sh`)

Desglose línea por línea

- `if ! command -v strace >/dev/null 2>&1; then ... exit 1; fi`: verifica que `strace` esté instalado. Si no lo está, muestra cómo instalarlo y termina con `exit 1`. La parte `>/dev/null 2>&1` hace que la verificación no imprima nada en pantalla.
- `chmod +x ./sync_service.sh`: le da permiso de ejecución al servicio para poder correrlo.
- `strace -f -e trace=file -o auditoria_archivos.log ./sync_service.sh`: ejecuta el servicio a través de `strace`:
  - `-f`: también registra los procesos hijos que cree el script.
  - `-e trace=file`: registra solamente las llamadas al sistema relacionadas con archivos.
  - `-o auditoria_archivos.log`: guarda todo el registro en ese archivo en lugar de mostrarlo en pantalla.
- `grep "transactions.dat" auditoria_archivos.log`: busca dentro del registro únicamente las líneas que mencionan `transactions.dat`. Así aparece en pantalla la llamada exacta y la respuesta del Kernel.
- `strace -c -e trace=execve,openat,read,write,close,newfstatat,faccessat2 ./sync_service.sh`: vuelve a ejecutar el servicio con `-c`, que en lugar de mostrar cada llamada arma una tabla con cuántas veces se usó cada una, cuántas fallaron y cuánto tiempo consumieron.

En resumen

Este script corre el servicio "vigilado" por `strace`, guarda todo lo que el servicio le pidió al Kernel sobre archivos, filtra la línea de `transactions.dat` para mostrar el error real y, por último, presenta una tabla estadística de las llamadas al sistema. La línea clave que aparece es:

```
text
faccessat2(AT_FDCWD, "/etc/secure_app/transactions.dat", R_OK, AT_EACCESS) = -1 ENOENT (No
such file or directory)
```

Es decir, el Kernel responde que **el archivo no existe** (`ENOENT`).

Código `reparar_servicio.sh` (la solución)

(Insertar captura de `reparar_servicio.sh`)

Desglose línea por línea

- **`sudo mkdir -p /etc/secure_app`** : crea la carpeta que faltaba. Se usa `sudo` porque `/etc` solo puede modificarlo el administrador; `-p` evita errores si la carpeta ya existe.
- **`sudo bash -c 'cat << EOF > /etc/secure_app/transactions.dat ... EOF'`** : crea el archivo `transactions.dat` y escribe dentro tres transacciones de prueba. Todo lo que está entre `<< EOF` y `EOF` es el contenido que se guarda.
- **`sudo chmod 644 /etc/secure_app/transactions.dat`** : asigna permisos `644` : el dueño puede leer y escribir, y el resto de los usuarios solo puede leer (necesario para que el servicio pueda leerlo).
- **`ls -l /etc/secure_app/`** : muestra el archivo creado con sus permisos (`-rw-r--r--`).
- **`./sync_service.sh`** y **`echo "Codigo de salida: $?"`** : vuelve a ejecutar el servicio y muestra su código de salida. `$?` guarda el código del último comando ejecutado; ahora debe ser `0` .
- **`strace -f -e trace=file -o auditoria_ok.log ./sync_service.sh > /dev/null`** y **`grep "transactions.dat" auditoria_ok.log`** : repite la auditoría para confirmar que ahora el Kernel responde `= 0` (éxito) en lugar de `-1 ENOENT` .

En resumen

El script crea la carpeta y el archivo que el servicio no encontraba, le da permisos de lectura, vuelve a ejecutar el servicio (que ahora muestra las transacciones y termina con código `0`) y confirma con `strace` que el Kernel ya no devuelve error.

Código `restaurar_falla.sh` (para repetir la demostración)

(Insertar captura de `restaurar_falla.sh` si se usa)

Desglose línea por línea

- `sudo rm -rf /etc/secure_app`: borra la carpeta y el archivo creados por la solución, dejando el servidor como estaba al principio.
- `rm -f auditoria_archivos.log auditoria_ok.log`: borra los registros de auditorías anteriores.
- `./sync_service.sh` y `echo "Codigo de salida: $?"`: comprueba que el servicio vuelve a fallar con código `101`.

Cierre de la Unidad 1

Con `audit_syscalls.sh` demostramos que el "fallo crítico en el motor transaccional" no era un error del software ni de la base de datos: al evaluar `[ ! -r "$DATA_FILE" ]`, el programa pasa de Modo Usuario a Modo Kernel mediante la llamada al sistema `faccessat2`, y el Kernel responde `-1 ENOENT`, es decir, que el archivo `/etc/secure_app/transactions.dat` no existe. Lo descubrimos sin modificar el código del servicio, solo observando sus llamadas al sistema con `strace`. Finalmente, con `reparar_servicio.sh` creamos la carpeta y el archivo con permisos `644`, y comprobamos que el servicio finaliza con código `0` y que el Kernel ahora responde `= 0`.

Unidad 2 — Concurrencia, Condición de Carrera y Exclusión Mutua

Introducción breve

En la billetera virtual **Fintech Global Wallet**, el motor de transacciones corre en un servidor Red Hat usando varios hilos en paralelo. Durante un evento de alta demanda se detectaron dos anomalías:

1. **En memoria:** 8 hilos debían acreditar 250.000 depósitos de `$1` cada uno en una misma cuenta (total esperado: 2.000.000), pero el saldo final siempre era menor y cambiaba en cada ejecución (por ejemplo, 439.877).
2. **En un archivo:** dos procesos escribían el estado de una orden en el mismo archivo y uno borraba el dato del otro.

En ningún caso el sistema mostró errores: todos los procesos terminaron con código `0`, pero los datos quedaron corruptos.

Temas expuestos

- **Condición de carrera (*race condition*):** ocurre cuando varios hilos o procesos modifican el mismo dato al mismo tiempo sin coordinarse. El resultado depende de qué hilo llegue primero.
- **Operación no atómica:** `contador_compartido++` parece una sola instrucción, pero el procesador la hace en tres pasos (leer el valor, sumarle 1 y guardarlo). Si dos hilos leen el mismo valor antes de que el otro guarde, una de las sumas se pierde.
- **Sección crítica:** es la parte del código donde se accede al dato compartido.
- **Exclusión mutua (Mutex):** un cerrojo que garantiza que solo un hilo a la vez entre a la sección crítica; los demás esperan su turno.
- **Costo de la sincronización (*overhead*):** el Mutex da resultados correctos pero hace el programa más lento, porque el Kernel interviene para dormir y despertar a los hilos que esperan.

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./demostracion_tpo.sh`	`carrera` da un saldo incorrecto; `seguro` (la solución con Mutex) da 2.000.000; `time` muestra el costo del Mutex.
2	`./carrera_escritura.sh`	En el archivo solo queda el dato del último proceso que escribió (el problema en disco).

Código `demostracion_tpo.sh`

(Insertar captura de `demostracion_tpo.sh`)

Desglose línea por línea

- `if ! command -v gcc >/dev/null 2>&1; then ... exit 1; fi`: verifica que el compilador de C (`gcc`) esté instalado; si no, indica cómo instalarlo y termina.
- `gcc carrera.c -o carrera -lpthread` y `gcc seguro.c -o seguro -lpthread`: compilan los dos programas en C. `-o` define el nombre del ejecutable y `-lpthread` agrega la librería de hilos de Linux, necesaria para crear hilos y usar Mutex.
- `if [ $? -ne 0 ]; then ... exit 1; fi`: `$?` guarda el resultado del último comando; si es distinto de `0` (`-ne 0`), la compilación falló y el script se detiene.
- `./carrera`: ejecuta la versión sin protección; el saldo obtenido es menor a 2.000.000.

- `./seguro`: ejecuta la versión con Mutex; el saldo obtenido es exactamente 2.000.000.
- `time ./carrera` y `time ./seguro`: vuelven a ejecutar ambos midiendo el tiempo. `real` es el tiempo total, `user` el tiempo en Modo Usuario y `sys` el tiempo en Modo Kernel (donde se nota el costo del Mutex).

En resumen

El script compila los dos programas, ejecuta primero el que tiene la condición de carrera y después el que está protegido con Mutex para comparar los saldos, y finalmente mide el tiempo de ambos para mostrar que la versión segura es correcta pero más lenta.

Código `carrera.c` (el problema en memoria)

(Insertar captura de `carrera.c`)

Desglose línea por línea

- `#include <pthread.h>`: incluye la librería de hilos.
- `#define NUM_HILOS 8` y `#define ITERACIONES 250000`: se crean 8 hilos y cada uno hace 250.000 depósitos (8 × 250.000 = 2.000.000).
- `long contador_compartido = 0;`: variable global que representa el saldo; todos los hilos usan la misma.
- `void* depositar(void* arg)`: función que ejecuta cada hilo. Repite 250.000 veces `contador_compartido++` sin ninguna protección, por lo que los hilos se pisan entre sí.
- `pthread_create(&hilos[i], NULL, depositar, NULL);`: crea los 8 hilos, todos ejecutando `depositar`.
- `pthread_join(hilos[i], NULL);`: el programa principal espera a que terminen los 8 hilos.
- `printf("[RACE CONDITION] Total esperado: ... | Total obtenido: ...")`: muestra el saldo esperado y el obtenido.

Código `seguro.c` (la versión corregida con Mutex)

(Insertar captura de `seguro.c`)

Desglose línea por línea

- `pthread_mutex_t cerrojo;`: declara el Mutex que protege el saldo.
- `pthread_mutex_init(&cerrojo, NULL);`: inicializa el Mutex antes de crear los hilos.

- `pthread_mutex_lock(&cerrojo);` (sección de entrada): el hilo pide el cerrojo; si otro hilo lo tiene, espera.
- `contador_compartido++;` (sección crítica): con el cerrojo tomado, solo este hilo modifica el saldo.
- `pthread_mutex_unlock(&cerrojo);` (sección de salida): libera el cerrojo para el siguiente hilo.
- `pthread_mutex_destroy(&cerrojo);`: libera el Mutex al final, cuando ya terminaron todos los hilos.

Código `carrera_escritura.sh` (el problema en el archivo)

(Insertar captura de `carrera_escritura.sh`)

Desglose línea por línea

- `ARCHIVO_COMPARTIDO="recurso_compartido.txt"` y `echo "VALOR_INICIAL" > "$ARCHIVO_COMPARTIDO"`: crea el archivo compartido con el texto `VALOR_INICIAL`.
- `funcion_proceso1()` y `funcion_proceso2()`: cada función espera un tiempo distinto (`sleep 0.05` y `sleep 0.08`) y luego escribe su nombre en el archivo usando `>`, que **reemplaza** todo el contenido anterior.
- `funcion_proceso1 &` y `funcion_proceso2 &`: el `&` ejecuta ambas funciones en segundo plano y al mismo tiempo. `$!` guarda el número de proceso (PID) de cada una.
- `wait $PID1 $PID2`: espera a que ambos procesos terminen.
- `echo "Valor final retenido en el recurso: $(cat $ARCHIVO_COMPARTIDO)"`: muestra que en el archivo solo quedó el dato del último proceso que escribió.

En resumen

Dos procesos escriben al mismo tiempo en el mismo archivo sin coordinarse y usando `>`, que reemplaza el contenido. El resultado es que el último en escribir borra el dato del anterior sin que el sistema muestre ningún error.

Cierre de la Unidad 2

Demostramos que cuando varios hilos o procesos modifican un mismo recurso sin coordinación se produce una condición de carrera: el saldo queda por debajo de 2.000.000 y, en el archivo, un proceso borra el dato del otro, todo sin que el sistema informe errores. La solución es la exclusión mutua: con un Mutex (`pthread_mutex_lock` / `pthread_mutex_unlock`) el saldo da exactamente 2.000.000 en todas las ejecuciones, y con `flock` y `>>` ambos procesos conservan su dato en el archivo. También comprobamos con `time` que esta seguridad tiene un costo: la versión con Mutex consume más

tiempo en Modo Kernel (`sys`), porque el sistema operativo debe dormir y despertar a los hilos que esperan el cerrojo.

Unidad 3 — Asignación de Memoria Contigua y Fragmentación Externa

Introducción breve

En **Logistics Analytics Corp**, un servidor Red Hat ejecuta una base de datos que necesita reservar bloques **contiguos** de memoria RAM. La memoria libre está dividida en 5 huecos (100, 500, 200, 300 y 600 KB = 1.700 KB). Llegan 4 procesos que piden 212, 417, 112 y 426 KB. El sistema rechaza al proceso P4 (426 KB) por "memoria insuficiente", aunque después de ubicar a los otros tres quedan **959 KB libres**. Infraestructura culpaba a desarrollo por fugas de memoria y desarrollo decía que el sistema informaba mal el espacio libre.

Temas expuestos

- **Asignación contigua:** cada proceso debe ocupar un único bloque continuo de memoria.
- **Fragmentación externa:** hay memoria libre suficiente en total, pero está repartida en huecos chicos separados y ninguno alcanza por sí solo.
- **First-Fit (primer ajuste):** asigna el primer hueco donde entra el proceso. Es rápido, pero puede "romper" huecos grandes que después hacen falta.
- **Best-Fit (mejor ajuste):** revisa todos los huecos y elige aquel donde el proceso entra más justo, dejando libres los huecos grandes.
- **Paginación:** la solución que usa Linux en la realidad: divide la memoria en páginas de tamaño fijo (4096 bytes) para que un proceso no necesite un bloque contiguo.

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./ejecutar_tpo.sh`	Compila `simulador_memoria.c`, muestra el problema con First-Fit (P4 queda `RECHAZADO` con 959 KB libres), la solución con Best-Fit (entran los 4 procesos) y compara los tiempos de ambos algoritmos.

Código `ejecutar_tpo.sh` (Unidad 3)

(Insertar captura de `unidad_3/ejecutar_tpo.sh`)

Desglose línea por línea

- `if ! command -v gcc ...; fi`: verifica que `gcc` esté instalado.
- `gcc simulador_memoria.c -o simulador_memoria`: compila el simulador en C.
- `if [ $? -ne 0 ]; then ... exit 1; fi`: si la compilación falló, detiene el script.
- `./simulador_memoria F`: ejecuta el simulador con la letra `F`, que le indica usar First-Fit (reproduce el problema).
- `./simulador_memoria B`: ejecuta el simulador con la letra `B`, que le indica usar Best-Fit (muestra la solución).
- `time ./simulador_memoria F > /dev/null` y `time ./simulador_memoria B > /dev/null`: miden el tiempo de cada algoritmo. `> /dev/null` descarta el texto del simulador para que solo se vea la medición.

En resumen

El script compila el simulador, lo ejecuta primero con First-Fit (donde aparece el rechazo de P4) y después con Best-Fit sobre los mismos datos (donde entran los 4 procesos), y finalmente compara el tiempo que tarda cada algoritmo.

Código `simulador_memoria.c`

(Insertar captura de `simulador_memoria.c`)

Desglose bloque por bloque

- **Validación del parámetro (`argc`, `argv[1][0]`):** el programa recibe la letra del algoritmo. Si no se pasa ninguna o no es `F` ni `B`, muestra cómo usarlo y termina con error.

- **Datos iniciales:** `huecos[5] = {100, 500, 200, 300, 600}` son los huecos libres en KB; `procesos[4] = {212, 417, 112, 426}` son los KB que pide cada proceso; `asignacion[4]` empieza en `-1` y guarda en qué hueco quedó cada proceso (`-1` = rechazado).
- **Estado inicial:** recorre los arreglos y muestra la memoria libre total (1.700 KB) y la memoria pedida (1.167 KB).
- **First-Fit:** para cada proceso recorre los huecos en orden; en el primero donde entra (`huecos[j] >= procesos[i]`) lo asigna, descuenta ese espacio y corta la búsqueda con `break`.
- **Best-Fit:** para cada proceso recorre **todos** los huecos y se queda con el más chico donde entra (`mejor_idx`). Recién al terminar de revisar todos, lo asigna y descuenta el espacio.
- **Resultados:** muestra una tabla con el hueco asignado a cada proceso (o `RECHAZADO (Sin espacio contiguo)`) y suma la memoria libre que quedó.

Cierre de la Unidad 3

Con First-Fit, P1 (212 KB) ocupa el Hueco 2 de 500 KB, P2 (417 KB) ocupa el Hueco 5 de 600 KB y P3 (112 KB) vuelve a usar el Hueco 2. Cuando llega P4 (426 KB) ningún hueco individual alcanza y es rechazado, aunque quedan 959 KB libres en total: esto es la fragmentación externa. Con Best-Fit, cada proceso usa el hueco más justo (P1 en el Hueco 4, P2 en el 2, P3 en el 3) y se conserva el Hueco 5 de 600 KB, donde entra P4; los cuatro procesos quedan asignados. De todas formas, Best-Fit solo mejora el problema: los sistemas actuales como Red Hat lo eliminan con paginación, porque un proceso puede repartirse en páginas de 4096 bytes que no necesitan estar juntas.

Unidad 4 — Sistema de Archivos: Asignación Contigua vs. Inodos

Introducción breve

En **CloudVault Data Solutions**, un disco fue configurado con **asignación contigua**: cada archivo debe ocupar bloques seguidos. El disco tiene 5 huecos libres (100, 500, 200, 300 y 600 bloques). Al guardar 4 archivos de respaldo (212, 417, 112 y 426 bloques), el cuarto (`backup_transaccional.iso`) falla con el error `No space left on device (ENOSPC)`, aunque quedan **959 bloques libres**. Además, un archivo de registro existente no podía crecer porque el bloque siguiente estaba ocupado por otro archivo.

Temas expuestos

- **Asignación contigua en disco:** el archivo debe ocupar bloques consecutivos; sufre fragmentación externa y no permite que los archivos crezcan con facilidad.
- **Inodos (asignación indexada):** en Linux (`ext4` / `XFS`) cada archivo tiene un inodo, una ficha con sus datos (permisos, dueño, tamaño, fechas) y la lista de bloques donde está guardado. Los bloques pueden estar en cualquier parte del disco.
- `ls -li` **y** `stat` **:** muestran el número de inodo de un archivo y su información.
- **Número mágico (** `file` **):** Linux identifica el tipo de archivo leyendo sus primeros bytes, no por su extensión.

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./ejecutar_tpo.sh`	Compila `simulador_filesystem.c` , muestra el problema con asignación contigua First-Fit (el Archivo 4 da `NO ENTRA`), prueba Best-Fit, muestra la solución real con Inodos (entran todos repartidos en distintos huecos) e inspecciona un inodo real con `ls -li` , `stat` y `file` .

Código `ejecutar_tpo.sh` (Unidad 4)

(Insertar captura de `unidad_4/ejecutar_tpo.sh`)

Desglose línea por línea

- `if ! command -v gcc ...; fi` : verifica que `gcc` esté instalado.
- `gcc simulador_filesystem.c -o simulador_filesystem` : compila el simulador del disco.
- `./simulador_filesystem F` : asignación contigua con First-Fit; el Archivo 4 no entra (el problema).
- `./simulador_filesystem B` : asignación contigua con Best-Fit; los archivos entran porque cada uno usa el hueco más justo.
- `./simulador_filesystem I` : asignación con inodos (la solución de Linux); cada archivo se reparte en varios huecos libres.
- `ls -li simulador_filesystem.c` : `-i` muestra en la primera columna el número de inodo real del archivo.

- `stat simulador_filesystem.c`: muestra la información del inodo: número de inodo, tamaño, bloques ocupados, tamaño de bloque (`IO Block: 4096`), permisos, dueño y fechas.
- `file simulador_filesystem.c` y `file simulador_filesystem`: identifican el tipo real de cada archivo: el primero es código fuente en texto y el segundo un ejecutable de Linux (`ELF 64-bit`).

En resumen

El script compila el simulador y lo ejecuta en los tres modos para comparar el problema de la asignación contigua frente a la solución por inodos. Luego usa `ls -li`, `stat` y `file` para mostrar cómo Linux guarda la información de un archivo real en su inodo y cómo reconoce su tipo.

Código `simulador_filesystem.c`

(Insertar captura de `simulador_filesystem.c`)

Desglose bloque por bloque

- **Datos iniciales:** `huecos[5] = {100, 500, 200, 300, 600}` son los bloques libres del disco y `archivos[4] = {212, 417, 112, 426}` el tamaño de cada archivo en bloques.
- **Modos `F` y `B` (contiguos):** un archivo solo se guarda si entra completo en un único hueco (`huecos[j] >= archivos[i]`). `F` usa el primer hueco que sirve y `B` el más justo.
- **Modo `I` (inodos):** para cada archivo guarda en `falta_guardar` cuántos bloques necesita. Recorre los huecos y toma los bloques libres que haya en cada uno, aunque no alcancen para todo el archivo (`usar = (huecos[j] >= falta_guardar) ? falta_guardar : huecos[j]`). Sigue con el próximo hueco hasta completar el archivo y lo marca como guardado (`asignacion[i] = 99`).
- **Resultados:** muestra el estado de cada archivo y los bloques libres que quedaron.

Cierre de la Unidad 4

Con asignación contigua, el Archivo 4 (426 bloques) no puede guardarse aunque quedan 959 bloques libres, porque ningún hueco individual alcanza: es la fragmentación externa en disco. Con inodos, el sistema de archivos reparte los bloques de un mismo archivo en distintos huecos libres y anota en el inodo dónde está cada parte, por lo que los cuatro archivos se guardan y pueden crecer sin necesidad de moverse en el disco. Con `ls -li`, `stat` y `file` también vimos cómo Linux guarda la información de cada archivo en su inodo y cómo reconoce su tipo por el número mágico.

Unidad 5 — Entrada/Salida y Uso de Búfer

Introducción breve

En **AeroData Telemetry Corp**, un servicio escrito en C guarda datos en disco en un servidor Red Hat. Tras ponerlo en producción, el servidor se volvió lento y el procesador pasaba la mayor parte del tiempo en Modo Kernel. La causa es que el programa escribe **de a 1 byte por vez**: para guardar 100.000 caracteres hace 100.000 llamadas al sistema `write()`. Se busca demostrar la diferencia con una versión que usa un **búfer** de 4.000 bytes.

Temas expuestos

- **Costo de cada llamada al sistema:** cada `write()` obliga al procesador a pasar de Modo Usuario a Modo Kernel y volver. Hacerlo miles de veces para mover muy pocos datos genera una gran pérdida de tiempo.
- **Búfer (*buffering*):** un espacio en memoria donde se juntan los datos para enviarlos al Kernel todos juntos, en pocas llamadas.
- **Medición con `time` y `strace -c`:** `time` mide el tiempo en Modo Kernel (`sys`) y `strace -c` cuenta cuántas llamadas al sistema se hicieron.

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./ejecutar_tpo.sh`	Compila ambos programas, mide el tiempo de `sin_bufer` (el problema) y `con_bufer` (la solución) y cuenta con `strace -c` las llamadas a `write` (100.001 frente a 26).

Código `ejecutar_tpo.sh` (Unidad 5)

(Insertar captura de `unidad_5/ejecutar_tpo.sh`)

Desglose línea por línea

- `if ! command -v gcc ... || ! command -v strace ...; then ... fi`: verifica que estén instalados `gcc` y `strace`. El `||` significa "o": si falta cualquiera de los dos, se detiene.

- `gcc sin_bufer.c -o sin_bufer && gcc con_bufer.c -o con_bufer`: compila los dos programas. El `&&` hace que el segundo solo se compile si el primero salió bien.
- `time ./sin_bufer` y `time ./con_bufer`: ejecutan ambos y miden su tiempo, en especial el `sys` (Modo Kernel).
- `strace -c -e trace=write ./sin_bufer > /dev/null` y lo mismo con `./con_bufer`: cuentan solo las llamadas a `write` de cada programa.
- `rm -f salida_lenta.txt salida_rapida.txt`: borra los archivos generados en la prueba.

En resumen

El script compila las dos versiones, mide cuánto tarda cada una y cuenta con `strace` cuántas veces cada programa llamó a `write`, mostrando en una sola ejecución el problema de escribir de a un byte y la solución de escribir con búfer.

Código `sin_bufer.c` (el problema)

(Insertar captura de `sin_bufer.c`)

Desglose línea por línea

- `#include <fcntl.h>` y `#include <unistd.h>`: incluyen las funciones `open`, `write` y `close`, que llaman directamente al Kernel.
- `int fd = open("salida_lenta.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);`: abre el archivo para escribir (`O_WRONLY`), lo crea si no existe (`O_CREAT`) y lo vacía si ya tenía contenido (`O_TRUNC`). Guarda en `fd` el descriptor del archivo.
- `if (fd < 0) return 1;`: si no se pudo abrir, termina con error.
- `for (...100000...) { write(fd, "A", 1); }`: escribe la letra `A` de a 1 byte, 100.000 veces: son 100.000 llamadas al Kernel.
- `close(fd);`: cierra el archivo.

Código `con_bufer.c` (la solución)

(Insertar captura de `con_bufer.c`)

Desglose línea por línea

- `char bufer[4000];` y `memset(bufer, 'A', 4000);`: crea un búfer de 4.000 bytes en memoria y lo llena con letras `A`, sin intervención del Kernel.

- `for (int i = 0; i < 25; i++) { write(fd, bufer, 4000); }`: envía el búfer completo 25 veces (25 × 4.000 = 100.000 bytes): solo 25 llamadas al Kernel.
- `close(fd);`: cierra el archivo.

Cierre de la Unidad 5

Las dos versiones escriben exactamente el mismo archivo de 100.000 bytes, pero la versión sin búfer hace 100.001 llamadas a `write` (100.000 por las letras y una por el mensaje final), mientras que la versión con búfer hace solo 26 (25 bloques y una por los mensajes). Esto representa un 99,97 % menos de pasajes entre Modo Usuario y Modo Kernel, y el tiempo `sys` baja prácticamente a cero. La conclusión es que en Entrada/Salida no conviene pedirle al Kernel muchas operaciones pequeñas: juntar los datos en un búfer y escribirlos en bloques reduce drásticamente el costo.

Unidad 6 — Seguridad: Firewall, Contraseñas, ACL e Integridad

Introducción breve

Una empresa financiera ejecuta todos los días en un servidor Red Hat el script `pagos_empresa.sh`, que realiza la transferencia de sueldos. Un intruso (o un virus) logró modificar el archivo agregándole una línea maliciosa, manteniendo el mismo nombre. Si el administrador lo ejecuta confiando solo en el nombre, el código malicioso se ejecuta. Además, la empresa necesita controlar qué puertos de red están abiertos, guardar las contraseñas de forma segura y dar acceso a un reporte confidencial solo a un usuario auditor.

Temas expuestos

- **Firewall (`firewall-cmd`):** permite abrir o bloquear puertos para controlar qué conexiones de red se aceptan.
- **Contraseñas con Hash y Salt (`/etc/shadow`):** Linux no guarda las contraseñas, sino un resumen irreversible (hash) combinado con un valor aleatorio (salt), para que no puedan descubrirse aunque se lea el archivo.

- **Listas de Control de Acceso (ACL):** con `setfacl` y `getfacl` se puede dar permiso a un usuario puntual sobre un archivo sin abrirlo al resto.
- **Integridad con Hash SHA-256 (`sha256sum`):** se calcula una "huella digital" del archivo; si se modifica aunque sea un carácter, la huella cambia completamente y se detecta la alteración.
- **Defensa en profundidad:** combinar varias capas de protección (red, usuarios, permisos e integridad).

Orden de ejecución en la presentación

Paso	Comando	Qué se muestra
1	`./ejecutar_tpo.sh`	Prueba conectividad y firewall entre `servera` (puerto 8080 bloqueado) y `serverb` (puerto 8080 abierto), crea el usuario `auditor` mostrando su contraseña como Hash + Salt en `/etc/shadow`, le otorga acceso de lectura exclusivo con ACL (`setfacl` / `getfacl`) y demuestra cómo el comprobador de integridad (`sha256sum -c`) detecta y bloquea la modificación maliciosa de `pagos_empresa.sh` .

Código `pagos_empresa.sh` (el script protegido)

(Insertar captura de `pagos_empresa.sh`)

Desglose línea por línea

- `echo "[SISTEMA DE PAGOS]: Conectando con el banco..."` y `echo "[SISTEMA DE PAGOS]: Transferencia de sueldos realizada con exito."` : simulan la conexión con el banco y la transferencia de sueldos. Es el archivo crítico de la empresa que el intruso modifica y que debemos proteger.

Código `ejecutar_tpo.sh` (Unidad 6)

(Insertar captura de `unidad_6/ejecutar_tpo.sh`)

Desglose bloque por bloque

- **1. Conectividad y Firewall entre servidores (`servera` y `serverb`):**
  - `ping -c 2 servera` y `ping -c 2 serverb` : envían 2 paquetes a cada servidor para comprobar que ambos responden por red.

- `ssh student@servera "... firewall-cmd --remove-port=8080/tcp ..."`: se conecta por SSH a `servera` y bloquea el puerto 8080 en su firewall.
- `ssh student@serverb "... firewall-cmd --add-port=8080/tcp ...; nohup python3 -m http.server 8080 ... &"`: se conecta a `serverb`, abre el puerto 8080 en su firewall y levanta un servidor web en ese puerto.
- `curl -s --connect-timeout 2 http://servera:8080` y `http://serverb:8080`: intenta conectarse a ambos desde afuera. `servera` rechaza la conexión (`[BLOQUEADO]`) y `serverb` la permite (`[PERMITIDO]`).

- **2. Creación del usuario `auditor` y contraseña con Hash + Salt en `/etc/shadow`:**
  - `sudo useradd -m auditor`: crea el usuario `auditor` junto con su carpeta personal (`/home/auditor`).
  - `echo "auditor:RedHat2026" | sudo chpasswd`: le asigna la contraseña `RedHat2026`.
  - `sudo grep "^auditor:" /etc/shadow`: muestra la línea del usuario en `/etc/shadow`, comprobando que Linux nunca guarda la contraseña en texto plano sino como un resumen irreversible (**Hash + Salt**).
- **3. Lista de Control de Acceso (ACL) para el usuario `auditor`:**
  - `chmod 600 $ARCHIVO`: cierra el reporte confidencial para que solo su dueño tenga permiso; el grupo y el resto quedan sin acceso.
  - `setfacl -m u:auditor:r-- $ARCHIVO`: agrega una regla ACL para darle permiso de solo lectura (`r--`) únicamente al usuario `auditor`, sin abrir el archivo a los demás usuarios.
  - `ls -l $ARCHIVO`, `getfacl $ARCHIVO` y `sudo su - auditor -c "cat $ARCHIVO"`: muestran el signo `+` de la ACL activa, el detalle de permisos y comprueban que `auditor` puede leer el reporte.
- **4. Integridad con Hash SHA-256 sobre `pagos_empresa.sh`:**
  - `chmod 750 pagos_empresa.sh` y `sha256sum pagos_empresa.sh > sumacomp.txt`: restringe los permisos del script de pagos y guarda su huella digital oficial (SHA-256) en `sumacomp.txt`.
  - `if sha256sum -c sumacomp.txt; then ./pagos_empresa.sh; fi`: verifica la huella antes de ejecutar. Como el archivo está sano, coincide (`OK`) y se realiza la transferencia.
  - `echo 'echo "[VIRUS]: Robando contrasenas..."' >> pagos_empresa.sh`: simula el problema: un intruso agrega una línea maliciosa al final del archivo manteniendo el mismo nombre.
  - **Segunda verificación con `sha256sum -c sumacomp.txt`**: como el archivo cambió, la huella SHA-256 es completamente distinta (`FAILED`), entra por el bloque `else`, muestra `[BLOQUEADO]` y **cancela la ejecución** antes de que corra el virus.

- `sed -i '/VIRUS/d' pagos_empresa.sh` y `rm -f sumacomp.txt`: restauran el script original y limpian el entorno.

En resumen

Este único script recorre las cuatro capas de seguridad de la unidad: primero muestra en red cómo el firewall bloquea el puerto 8080 en `servera` y lo permite en `serverb`; segundo, crea al usuario `auditor` y verifica que su contraseña se guarda cifrada con Hash y Salt en `/etc/shadow`; tercero, usa una ACL para darle lectura exclusiva sobre un archivo cerrado con `chmod 600`; y cuarto, demuestra cómo un comprobador de integridad con `sha256sum` detecta que `pagos_empresa.sh` fue modificado por un virus y bloquea su ejecución.

Cierre de la Unidad 6

Implementamos una defensa en profundidad sobre Red Hat en cuatro niveles dentro de una misma demostración: en la red, con `firewall-cmd` controlamos qué puertos aceptan conexiones entre servidores (`servera` bloqueado y `serverb` habilitado); en la autenticación, comprobamos en `/etc/shadow` que las contraseñas se almacenan como Hash con Salt; en los permisos, aplicamos ACL (`setfacl`) para que únicamente el usuario `auditor` pueda leer un reporte confidencial; y en la integridad, verificamos la huella SHA-256 de `pagos_empresa.sh` antes de correrlo. Así, cuando el archivo es alterado con una línea maliciosa, la huella deja de coincidir y su ejecución se bloquea automáticamente.